



Implementasi Protokol HTTPS dengan Kriptografi *Elliptic-Curve* pada Proses Pengiriman Data Kelembaban Tanah dengan IoT

^{[1]*}Youngky, ^[2]Syamsul Bahri, ^[3]Kasliono

^{[1][2][3]}Jurusan Rekayasa Sistem Komputer, Fakultas MIPA, Universitas Tanjungpura, Pontianak, Indonesia

e-mail: ^{[1]*}youngkyaja@student.untan.ac.id, ^[2]syamsul.bahri@siskom.untan.ac.id, ^[3]kasliono@siskom.untan.ac.id

Email Penulis Korespondensi: youngkyaja@student.untan.ac.id

Submitted: 04/03/2025; Accepted: 12/03/2025; Published: 25/04/2025

Abstrak

Dalam sistem penyiraman tanaman otomatis, data sensor kelembaban tanah memegang peran penting misalnya untuk menentukan kapan tanaman memerlukan air atau tidak. Permasalahan yang muncul dengan penerapan IoT adalah aspek keamanan yang sering terlewatkan. Setiap perangkat IoT seharusnya mengantisipasi bahwa perangkat tersebut mungkin akan diserang selama masa pemakaiannya. Penelitian yang akan dilaksanakan berfokus pada penerapan protokol HTTPS dengan algoritma enkripsi *Elliptic-Curve Cryptography (ECC)*, dalam proses transmisi data kelembaban tanah dengan perangkat IoT melalui internet. Tujuan dari penelitian ini adalah untuk mengamankan pengiriman data kelembaban tanah selama proses transmisi dari perangkat IoT menuju web server. Hal yang perlu diperhatikan dalam aspek keamanan pada proses transmisi data adalah bentuk data yang dikirimkan harus berupa ciphertext. Ciphertext sendiri merupakan bentuk data yang tidak dapat dibaca baik oleh manusia ataupun mesin, sehingga tidak dapat diperoleh informasi dari data tersebut kecuali data tersebut berhasil di dekripsi. Dengan menggunakan pendekatan brute force, hal ini akan menghabiskan waktu yang sangat lama. Dari pengujian yang dilaksanakan, dapat diperoleh hasil bahwa data kelembaban tanah yang dikirimkan ketika dibandingkan dengan data yang disimpan pada server, telah bernilai sama. Kemudian data dikirimkan dalam bentuk ciphertext sehingga informasi berupa tingkat kelembaban tanah dari data yang dikirimkan tidak dapat diperoleh ataupun diubah.

Kata kunci—HTTPS, *Elliptic-Curve Cryptography*, Ciphertext, Kelembaban Tanah, *Internet of Things*.

1. PENDAHULUAN

Seiring perkembangan mikrokontroler dan internet yang semakin pesat, *Internet of Things (IoT)* menjadi banyak diterapkan dalam kehidupan sehari-hari [1]. IoT merupakan sebuah perangkat keras yang memanfaatkan mikrokontroler untuk melakukan otomatisasi dengan mengambil, mengirim dan mengolah data melalui internet [2]. Perkembangan IoT ini mengakibatkan luasnya penerapan IoT pada masa sekarang, baik itu pada bidang industri, kesehatan, pertanian dan lainnya. Hal ini karena perangkat IoT dianggap mampu membantu manusia menyelesaikan masalah dengan berbagai solusi otomatisasi yang ditawarkan,

contohnya pada bidang agrikultur yaitu meningkatkan hasil panen dan mengurangi biaya operasional [3].

Namun terdapat permasalahan yang muncul dengan penerapan IoT, misalnya aspek keamanan yang sering terlewatkan. Perangkat IoT yang terjual pada umumnya tidak mampu menawarkan solusi keamanan yang baik. Hal ini dapat dilihat dari perangkat *smart home* seperti lampu, TV dan lainnya [4]. Salah satu hal signifikan yang membuat IoT menjadi teknologi yang rentan terhadap serangan *cyber* adalah kurangnya ketersediaan mekanisme keamanan yang layak [5]. Setiap perangkat IoT, sama halnya dengan perangkat keras lainnya

seharusnya mengantisipasi bahwa perangkat tersebut mungkin akan diserang selama masa pemakaiannya.

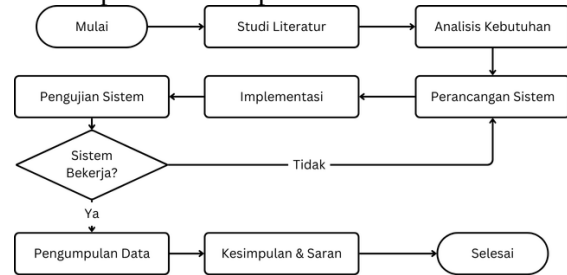
Dalam proses transmisi data, terdapat berbagai macam protokol yang dapat digunakan, salah satunya *Hypertext Transfer Protocol Secure* (HTTPS). HTTPS merupakan pengembangan dari HTTP, protokol ini menyediakan keamanan kriptografi, dengan membawa pesan HTTP melalui *Transport Layer Security* (TLS) [6]. Sebelum dapat berkomunikasi dengan HTTPS, langkah awal yang perlu dilakukan yaitu *TLS Handshake*. *TLS Handshake* merupakan proses komunikasi antara *client* dengan *server* untuk memperoleh *Session Key* yaitu *key* yang nantinya akan digunakan untuk melakukan enkripsi terhadap data yang akan dikirimkan [7]. Selain digunakan untuk mengamankan jalur komunikasi antara *client* dengan *web server*, protokol ini juga dimaksudkan untuk memastikan bahwa *client* dapat mempercayai bahwa *web server* yang sedang diakses adalah tujuan yang dimaksud oleh *client*, dikarenakan *web server* yang sedang diakses, telah terautentikasi dengan sertifikat digital yang valid [8].

Penelitian yang akan dilaksanakan berfokus pada penerapan protokol HTTPS dengan algoritma kriptografi *Elliptic-Curve* (ECC), dalam proses transmisi data perangkat IoT penyiraman tanaman otomatis melalui internet. Algoritma ini memungkinkan dua pihak untuk bertukar *public key* dengan ukuran *key* yang kecil, sehingga cocok untuk kondisi lingkungan dengan kekuatan komputasi yang terbatas [9]. Tujuan daripada penelitian ini adalah untuk mengamankan proses pengiriman data IoT ke *web server* sehingga data dikirimkan dalam bentuk *ciphertext* melalui proses enkripsi.

2. METODOLOGI PENELITIAN

Jenis penelitian ini merupakan jenis penelitian dengan pendekatan kualitatif. Penelitian kualitatif adalah penelitian yang bersifat deskriptif, sehingga hasil daripada penelitian kualitatif berbentuk deskriptif seperti tulisan atau lisan dari sebuah fenomena yang diamati [10]. Terdapat lima cara untuk memperoleh hasil pada penelitian kualitatif yaitu etnografi, studi kasus, *grounded theory*,

fenomenologi dan biografi [11]. Penelitian ini menerapkan metode penelitian studi kasus.



Gambar 1. Diagram Alir Penelitian

Seperti pada Gambar 1, penelitian ini dilaksanakan dengan melalui beberapa tahap yang perlu dilakukan untuk memperoleh hasil penelitian secara sistematis.

2.1 Studi Literatur

Tahapan studi literatur pada penelitian ini dilakukan dengan mengumpulkan, serta melakukan analisis terhadap informasi terkait teknik dan metode yang akan digunakan. Tujuan dari langkah ini adalah untuk memastikan penelitian yang dilakukan tetap berada dalam batasan penelitian yang telah ditentukan. Langkah ini dilakukan untuk memahami cara mengimplementasi sistem, alat, metode yang akan digunakan.

2.2 Analisis Kebutuhan

Langkah analisis kebutuhan dilakukan untuk memahami terkait kebutuhan apa saja yang harus dipenuhi dalam penelitian ini. Analisis kebutuhan juga akan berfokus kepada tujuan penelitian sehingga penelitian tetap terlaksana dalam lingkup penelitian yang telah ditentukan. Kemudian langkah ini dilakukan untuk menentukan apa saja yang dibutuhkan dalam mengembangkan sistem yang akan dibuat, mulai dari perangkat keras, perangkat lunak hingga kebutuhan lainnya seperti metode atau protokol yang diterapkan.

2.2.1 Kebutuhan Perangkat Keras

Berikut adalah kebutuhan perangkat keras yang diperlukan pada penelitian ini.

1. ESP32

Merupakan sebuah mikrokontroler oleh *Espressif Systems*. ESP32 adalah mikrokontroler dengan mikroprosesor *Xtensa dual-core 32-Bit* sebagai inti. ESP32 juga dilengkapi dengan modul Wi-Fi yang mendukung koneksi Wi-Fi: *802.11 b/g/n*.



Selain itu ESP32 juga sudah dilengkapi dengan *Bluetooth*: v4.2. Walaupun bentuknya yang kecil, mikrokontroler ini lebih unggul dari pendahulunya yaitu ESP8266 dari segi fitur dan kekuatan komputasi serta jumlah *memory* yang dimiliki [12].

2. Capacitive Soil Moisture Sensor

Sensor kelembaban tanah merupakan sensor untuk menentukan tingkat kelembaban pada tanah. Sensor ini bekerja menggunakan 2 buah pelat konduktor. Ketika sensor ini dimasukkan kedalam tanah, kedua pelat ini membentuk kapasitor dengan media tanah sebagai dielektrik. Kapasitansi dari kapasitor ini meningkat seiring dengan peningkatan kadar air pada tanah. Kemudian nilai perubahan kapasitansi ini diubah menjadi tolak ukur dari tingkat kelembaban pada tanah. Keunggulan dari tipe sensor kapasitif dibandingkan dengan tipe resistif adalah tingkat akurasi deteksi yang lebih tinggi, dan lebih tahan terhadap korosi [13].

2.2.2 Kebutuhan Perangkat Lunak

Adapun kebutuhan perangkat lunak dalam penelitian adalah sebagai berikut.

1. REST API

Representational State Transfer atau juga dikenal sebagai *RESTful web API*, merupakan sebuah arsitektur perangkat lunak yang digunakan pada aplikasi berbasis *web*. REST API menggunakan protokol komunikasi HTTP/HTTPS. Dalam REST API dikenal istilah *client* dan *server*, *client* merupakan pihak pengguna akhir yang bertujuan untuk mengakses data yang disediakan oleh API, sedangkan *server* merupakan pihak yang menyediakan API. Terdapat 4 *request* yang umum digunakan dalam REST API yaitu POST untuk menambahkan data, GET untuk mengambil data, PUT untuk mengubah data dan DELETE untuk menghapus data dari *server* [14].

2. Let's Encrypt

Let's Encrypt merupakan sebuah *Certificate Authority* atau otoritas penyedia sertifikat digital, untuk memungkinkan *web server* dapat menerapkan protokol HTTPS. Model ini disebut dengan *Trust Model Infrastructure* yang memungkinkan bahwa setiap entitas pada dunia internet dapat saling memvalidasi satu sama lain untuk membangun

kepercayaan, validasi ini dilakukan dalam bentuk sertifikasi [16]. *Let's Encrypt* dijalankan oleh *Internet Security Research Group* (ISRG). ISRG menjalankan *Let's Encrypt* dengan model bisnis *non profit* sehingga sertifikasi digital dapat diperoleh oleh siapapun. Tujuan ISRG menjalankan otoritas sertifikasi digital *Let's Encrypt* adalah untuk memperluas penerapan keamanan komunikasi data melalui internet khususnya pada *web*, dan mempermudah banyak pihak untuk mendapatkan sertifikat digital [15].

3. Certbot

Certbot adalah sebuah *open source tools* yang memungkinkan *server* untuk menginstal dan menggunakan sertifikat digital yang disediakan oleh *Let's Encrypt*. *Certbot* dibuat oleh *Electronic Frontier Foundation* (EFF) yang berasal dari *San Francisco*. Terdapat 2 fungsi utama dari tools ini yaitu untuk melakukan instalasi sertifikat digital oleh *Let's Encrypt* pada *web server*, kemudian memperbarui sertifikat secara otomatis ketika melewati batas waktu masa aktif sertifikat. Sebagai *open source tools*, hal ini berarti *source code* dari *Certbot* dapat diakses. Terdapat 2 algoritma kriptografi yang tersedia yaitu RSA dan ECC, secara *default* *Certbot* akan melakukan instalasi sertifikat pada *web server* dengan kriptografi ECC [15].

2.2.3 Kebutuhan Lainnya

Berikut adalah metode atau protokol yang diterapkan pada penelitian ini.

1. HTTPS

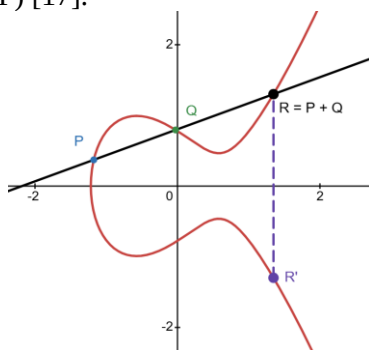
Hypertext Transfer Protocol Secure atau dikenal juga dengan singkatan HTTPS merupakan sebuah pengembangan dari protokol *Hypertext Transfer Protocol* (HTTP). HTTPS merupakan protokol komunikasi yang menerapkan enkripsi *Transport Layer Security* (TLS), yaitu sebuah protokol enkripsi yang dibuat berdasarkan pendahulunya *Secure Socket Layer* (SSL) [6]. Protokol enkripsi TLS/SSL mengamankan komunikasi dengan menggunakan infrastruktur *Asymmetric Public Key*, umumnya menggunakan algoritma yang disebut RSA (*Rivest-Shamir-Adleman*) dan ECC (*Elliptic-Curve Cryptography*). Protokol HTTPS (HTTP dengan SSL/TLS), merupakan standar untuk mengamankan banyak aspek di dunia internet seperti, mengunduh perangkat lunak, mengirim data penting, misalnya



memasukkan *password* dan autentikasi lainnya pada *website* [16].

2. Elliptic-Curve Cryptography

ECC adalah sebuah metode pada kriptografi yang merupakan bagian dari *Asymmetric Encryption*. ECC merupakan salah satu alternatif dari RSA, dikarenakan keunggulannya pada *key size* yang lebih kecil untuk memperoleh tingkat keamanan yang sama dengan RSA. Kekuatan kriptografi dari ECC berada pada tingkat kesulitan untuk menyelesaikan permasalahan matematis *Elliptic-Curve Discrete Logarithm Problem* (ECDLP) [17].



Gambar 2. Elliptic-Curve

Persamaan umum dari *Elliptic-Curve* yang digunakan untuk memperoleh kurva seperti pada Gambar 2, dapat dilihat pada Persamaan (1).

$$y^2 = x^3 + ax + b \text{ mod } q \quad (1)$$

Dari Persamaan (1), dapat ditentukan sebuah titik contohnya P dan Q, dengan menjumlahkan P dan Q dapat diperoleh titik R. Kemudian dengan mengambil titik pada sebrang R melalui sumbu x, dapat diperoleh titik R'. Titik R' kemudian dapat digunakan untuk mengulang operasi misalnya dengan menjumlahkan dengan titik P untuk memperoleh titik baru pada kurva contohnya titik S [9]. Operasi ini akan dilakukan berkali kali sebanyak *n* kali sehingga secara matematis dapat dijabarkan seperti pada Persamaan (2).

$$nP = S \quad (2)$$

Dengan *nP* adalah perkalian skalar $nP = P \cdot P \cdot P$ sebanyak *n* kali. Walaupun Persamaan (2) terlihat sederhana, tetapi dengan Persamaan (2), ketika misalnya diketahui nilai *P* dan *S*, secara komputasi sangat sulit untuk menemukan nilai *n*, sehingga satu satunya cara yang bisa dilakukan untuk meretas adalah dengan menebak nilai *n*, hal inilah yang disebut dengan ECDLP [9].

2.3 Perancangan Sistem

Tahap perancangan sistem meliputi dua bagian yaitu perancangan perangkat keras, dan perancangan perangkat lunak. Sistem IoT yang digunakan untuk menerapkan algoritma kriptografi pada pengiriman data akan menggunakan ESP32 yang dihubungkan dengan sensor kelembaban tanah. Kemudian sebelum data dikirimkan, data tersebut akan diubah menjadi *ciphertext* dengan algoritma ECC. Pada saat data tersebut diterima oleh *web server*, data tersebut kemudian akan disimpan pada *database* MySQL. Data ini kemudian dapat diakses oleh pengguna dengan melalui aplikasi *frontend* pada halaman *dashboard* yang telah disediakan. Aplikasi yang digunakan untuk mengakses data berbasis *web* sehingga dapat diakses dengan perangkat apapun selama memiliki *browser*.

2.4 Implementasi

Sistem yang dibuat akan menggunakan mikrokontroler ESP32, kemudian akan ditentukan batasan kelembaban tanah di antara 2 kategori, yaitu kondisi tanah kering dan tanah lembab melalui kalibrasi sensor. Data yang diperoleh oleh sistem akan dikirim ke *web server* dengan jeda waktu setiap 10 detik. Data kelembaban tanah akan dikirim dengan mengakses POST API, dan dienkripsi dengan algoritma ECC melalui HTTPS. Selain itu ESP32 juga akan mengambil data berisi konfigurasi pada tingkat persentase ke berapa sistem penyiraman akan dinyalakan, hal ini untuk mensimulasikan pengambilan data ke *server*.

2.5 Pengujian Sistem

Pada tahap ini sistem akan diuji berdasarkan fungsionalitas dan kinerja. Pengujian fungsionalitas dapat dilakukan dengan menguji proses dari pengambilan data sensor hingga data tersebut tersimpan pada *database*, dan data dapat ditampilkan pada halaman *dashboard*. Kemudian pengujian kinerja akan dilakukan dengan memeriksa apakah data benar dikirimkan dalam bentuk *ciphertext*, dan membandingkan apakah data yang diambil oleh sensor dengan data yang tampil pada *dashboard* telah sesuai. Adapun pada tahap pengujian sistem, akan ditinjau apakah sistem sudah bekerja sesuai dengan perancangan, dan apabila tidak maka sistem

akan dirancang kembali, untuk memastikan tercapainya tujuan penelitian.

2.6 Pengumpulan Data

Setelah melakukan implementasi dan pengujian maka tahap berikutnya pada penelitian ini adalah pengumpulan data, tahap ini dilakukan untuk membantu proses penarikan kesimpulan sekaligus menentukan hasil dari penelitian. Dalam sebuah penelitian terdapat dua tipe data yaitu data primer dan data sekunder. Data primer merupakan sebuah data yang dimana sumber dari data ini diperoleh secara langsung dari sumber sebenarnya [18]. Adapun data sekunder merupakan sebuah data yang dimana sumber dari data ini tidak diperoleh secara langsung, melainkan diperoleh sebagai data primer dari sumber yang diambil, sehingga pengguna daripada data sekunder tidak merasakan secara langsung fenomena yang sedang diteliti [19]. Maka berdasarkan tipe data dan sumber dari data yang telah dijabarkan maka data pada penelitian ini dapat dikategorikan sebagai data primer, karena data yang akan diperoleh pada penelitian ini tidak melalui sumber lain, melainkan langsung diperoleh ketika penelitian ini dilaksanakan. Oleh karena itu proses pengolahan data mulai dari pengumpulan data hingga analisis melalui tahapan tersendiri yaitu sebagai berikut.

2.6.1 Teknik Pengumpulan Data

Teknik pengumpulan data yang digunakan pada penelitian ini adalah observasi. Sebagai salah satu metode saintifik, teknik observasi harus dijalankan secara sistematis, dengan berfokus pada pertanyaan yang hendak diselesaikan, sehingga data yang dikumpulkan merupakan data yang dapat digunakan untuk menjawab permasalahan dalam penelitian [20]. Dengan demikian, maka langkah pengumpulan data dalam penelitian ini, yang dimana data diperoleh melalui instrumen aplikasi *Wireshark* untuk mengetahui aktivitas pada jaringan merupakan bagian dalam teknik observasi.

2.6.2 Teknik Analisis Data

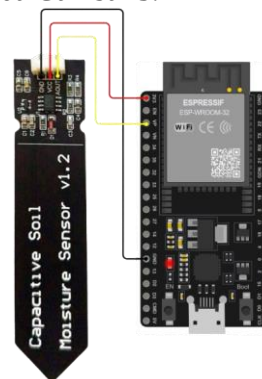
Teknik untuk melakukan analisis terhadap data yang diperoleh pada penelitian ini menerapkan teknik *content analysis*. *Content analysis* merupakan sebuah teknik analisis data yang dirancang untuk mengidentifikasi dan menginterpretasikan makna dengan

menggunakan potongan kecil data yang mewakili data secara keseluruhan dimana data ini dapat digunakan untuk menjelaskan sebuah fenomena yang terjadi [21]. Sebelum data dikumpulkan, akan dibagi dua kategori utama yaitu “terenkripsi” dan “tidak terenkripsi”. Dasar dalam kategori “terenkripsi” adalah bentuk data berupa *ciphertext*, sedangkan dasar pada kategori “tidak terenkripsi” adalah bentuk data yang merupakan *plaintext*. Setelah diperoleh data melalui teknik pengumpulan data, maka isi dari data ini akan dianalisis, untuk kemudian dapat dimasukkan dalam salah satu kategori yang telah ditentukan. Dari hasil analisis ini maka dapat dibuat kesimpulan sesuai dengan data yang telah diperoleh, untuk menjawab permasalahan yang telah ditentukan.

3. HASIL DAN PEMBAHASAN

3.1 Implementasi Perangkat Keras

Pada tahap implementasi perangkat keras, semua komponen perangkat keras yaitu ESP32 dan *Capacitive Soil Moisture Sensor* yang akan digunakan dalam penelitian ini disusun dan dihubungkan sesuai dengan *wiring diagram* pada Gambar 3.



Gambar 3. Wiring Diagram Perangkat Keras

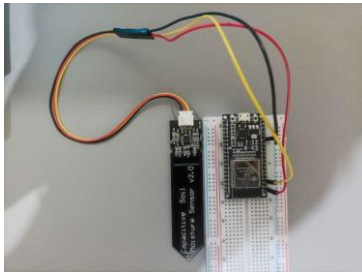
Untuk hubungan setiap pin pada wiring diagram perangkat keras dapat dilihat pada Tabel 1.

Tabel 1. Hubungan Pin Perangkat Keras

ESP32	Sensor
3V3	VCC
G36 (SP)	AOUT
GND	GND

Untuk hasil penerapan dari perangkat keras yang dibuat sesuai dengan *wiring*

diagram pada Gambar 3, dapat dilihat pada Gambar 4.



Gambar 4. Implementasi Perangkat Keras

Untuk dapat mendeteksi nilai dari kelembaban tanah, maka sensor kelembaban tanah perlu ditanamkan pada tanah. Tanah ini juga kemudian diberikan air secukupnya untuk mensimulasikan kondisi keadaan tanah yang lembab. Kemudian sensor akan dibiarkan pada tanah selama proses pengambilan data. Untuk gambar implementasi sensor pada tanah dapat dilihat pada Gambar 5.



Gambar 5. Sensor pada Tanah

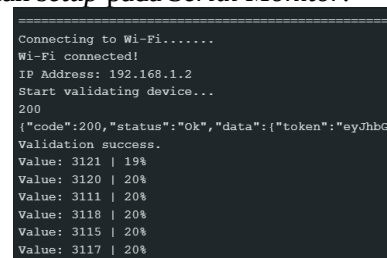
3.2 Implementasi Perangkat Lunak

Pembahasan implementasi perangkat lunak pada penelitian ini dibagi menjadi dua bagian yaitu implementasi perangkat lunak pada ESP32, dan implementasi perangkat lunak pada *web server*. Kemudian pembahasan implementasi pada *web server* juga dibagi terdiri dari tiga bagian, yaitu implementasi aplikasi *backend* yang berfungsi untuk menyediakan API yang digunakan ESP32, implementasi aplikasi *frontend* yang berfungsi untuk memonitor data yang diperoleh dari ESP32 dan implementasi pada *web server* secara keseluruhan.

3.2.1 Implementasi pada ESP32

Terdapat tiga proses utama dalam implementasi perangkat lunak pada ESP32, yaitu membaca data yang diperoleh oleh sensor kelembaban tanah, mengirimkan data kelembaban tanah, dan mengambil data konfigurasi nilai persentase kelembaban tanah

berapa, ESP32 akan menyiram tanaman, ketiga proses ini akan dijalankan pada *void loop* dengan jeda waktu selama 10 detik. Sebelum menjalankan proses *void loop* pada ESP32, pertama dilakukan proses *setup* terlebih dahulu pada ESP32. Proses *setup* ini dimulai dari menghubungkan dengan jaringan Wi-Fi dengan memasukkan SSID dan *password*. Kemudian sebelum dapat menggunakan API yang ada, ESP32 harus mendapatkan *token* JWT terlebih dahulu dengan menghubungi API validasi dimana API ini memerlukan ESP32 untuk mengirimkan kredensial berupa UUID dari ESP32 yang terdaftar pada *web server* dan *password* dari ESP32 tersebut. Setelah itu *token* akan disimpan dalam variabel yang kemudian dapat digunakan untuk menggunakan API lain. Untuk proses *setup* pada ESP32 dapat dilihat pada Gambar 6 yang menunjukkan hasil tampilan *setup* pada *Serial Monitor*.



Gambar 6. *Setup* pada ESP32

Setelah *setup* kemudian ESP32 akan menjalankan proses *void loop* yang dimulai dengan proses membaca data yang diperoleh oleh sensor kelembaban melalui *pin out* 36. Perlu diketahui bahwa data dari sensor berupa bilangan bulat dengan rentang dari 1600 hingga 3500, yang dimana nilai ini diperoleh dari hasil kalibrasi sensor. Angka 1600 diperoleh dengan memasukkan sensor ke dalam sebuah wadah berisi air, dan angka 3500 diperoleh ketika sensor dalam keadaan kering total. Dengan mengetahui nilai kalibrasi dari sensor, maka nilai ini dapat diubah ke bentuk persentase melalui fungsi *map()* pada *Arduino IDE*. Untuk contoh hasil data yang diperoleh dari sensor kelembaban tanah dapat dilihat pada Gambar 7 yang menunjukkan tampilan berisi data yang diambil oleh sensor misalnya 3439, beserta hasil konversi dalam bentuk persentase yaitu 3% yang menunjukkan kondisi sangat kering.

```
Value: 3439 | 3%
Value: 3446 | 2%
Value: 3445 | 2%
Value: 3445 | 2%
Value: 3466 | 1%
Value: 3454 | 2%
Value: 3472 | 1%
Value: 3478 | 1%
Value: 3470 | 1%
Value: 3463 | 1%
```

Gambar 7. Data Kelembaban Tanah

Proses pengiriman data pada ESP32 dilakukan melalui API *save-log* dengan metode POST. ESP32 akan mengirim data kelembaban berupa bilangan bulat ke *web server* dengan *token JWT* pada *request header*. Untuk hasil pengiriman data pada ESP32 dapat dilihat pada Gambar 8.

```
Value: 3130 | 19%
Start sending sensor data...
200
{"code":200,"status":"Ok","data":{"deviceId":2
Moisture data sent with status code 200
```

Gambar 8. Pengiriman Data ke Server

Proses mengambil data konfigurasi persentase pada ESP32 dilakukan melalui API *device-me* dengan metode GET. ESP32 akan mengirimkan GET *request* ke *web server* dengan menyertakan *token JWT* sebagai *request header*. Untuk hasil pengambilan data konfigurasi persentase kelembaban tanah pada ESP32 dapat dilihat pada Gambar 9.

```
Getting trigger percentage...
200
{"code":200,"status":"Ok","data":{"id":2
Trigger fetch success.
```

Gambar 9. Pengambilan Data oleh ESP32

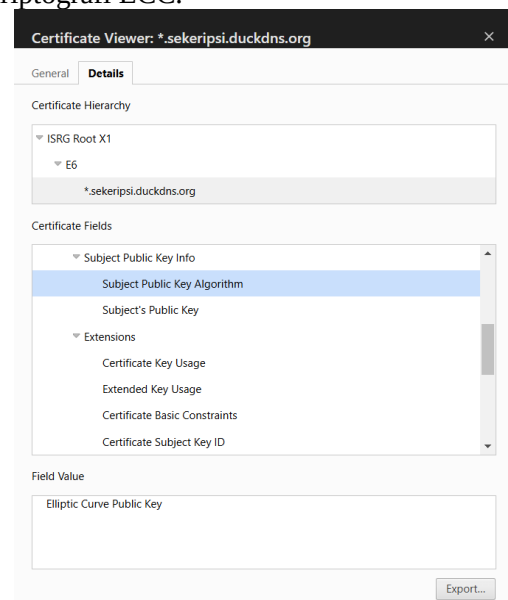
Ketiga proses yang berhubungan dengan *web server* baik itu validasi, mengirim data dan mengambil data, menghubungi *web server* melalui protokol HTTPS. Untuk kode bagaimana proses ini dilakukan dapat dilihat pada Gambar 10.

```
1 #include "SendLog.h"
2 #include "Certificate.h"
3
4 String sendLog(String token, int data) {
5   Serial.println("Start sending sensor data...");
6
7   String url = "https://nama-dns/api/save-log";
8   String jsonParameter;
9
10  HTTPClient http;
11
12  WiFiClientSecure client;
13  client.setCACert(rootCACertificate);
14
15  StaticJsonDocument<128> document;
16  StaticJsonDocument<64> requestBody;
```

```
17
18  requestBody["sensorData"] = data;
19
20  serializeJson(requestBody, jsonParameter);
21
22  http.begin(client, url);
23  http.addHeader("Content-Type", "application/json");
24  http.addHeader("Device-Token", token);
25
```

Gambar 10. Kode Penerapan HTTPS pada ESP32

Seperti pada Gambar 10, pertama diperlukan *url* dengan protokol HTTPS seperti pada baris ke 7 dalam Gambar 10. Kemudian ESP32 akan diatur sebagai *client* dengan *library WiFiClientSecure* seperti pada baris kode ke 12 dalam Gambar 10. Setelah itu perlu dimasukkan sertifikat *web server* untuk proses validasi *web server* yang digunakan untuk pengujian pada Tabel 2. Kemudian dengan menghubungi *web server* melalui protokol HTTP dengan url HTTPS, seperti pada baris kode ke 22 pada Gambar 10, maka ESP32 akan otomatis menghubungi *web server* melalui protokol HTTPS dengan konfigurasi pada *web server* yang menerapkan kriptografi ECC seperti pada Gambar 11, yang menunjukkan untuk berkomunikasi dengan *server*, perlu menggunakan protokol HTTPS dengan kriptografi ECC.



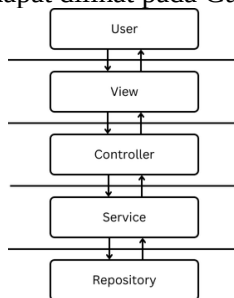
Gambar 11. Informasi Sertifikat pada Web Server

3.2.2 Implementasi pada Web Server

Pada implementasi *web server* terdapat dua aplikasi yang dibuat, yaitu aplikasi *backend* dan aplikasi *frontend*. Untuk aplikasi *backend*, sistem yang dibuat akan menerapkan arsitektur

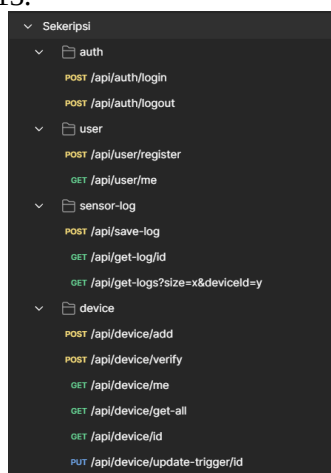


MVC (*Model-View-Controller*). Arsitektur MVC memungkinkan pembagian lapisan yang dimana setiap lapisan memiliki peran masing-masing. Tujuan dari penerapan arsitektur MVC adalah untuk menjaga integritas pada program *backend* sehingga dapat mudah dikembangkan jika diperlukan, dan pengguna tidak dapat secara langsung berinteraksi dengan *database* karena hanya *service* di *backend* yang dapat berinteraksi dengan *database*, sehingga hal ini dapat meningkatkan keamanan *database* pada *web server*. Arsitektur MVC yang diterapkan pada *server* dapat dilihat pada Gambar 12.



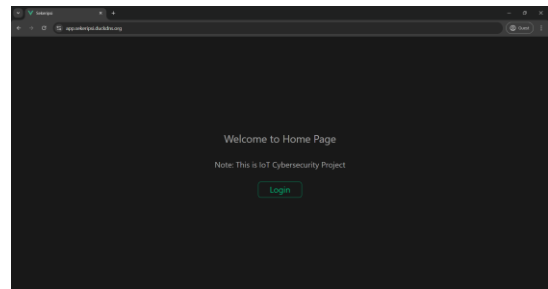
Gambar 12. Arsitektur MVC pada *Backend*

Web server yang dibuat mendukung beberapa API, untuk dapat digunakan oleh sistem yang dibangun. Daftar API yang dapat digunakan pada *web server* dapat dilihat pada Gambar 13.



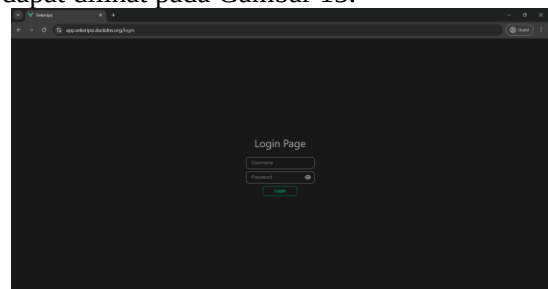
Gambar 13. Daftar API pada *Server*

Aplikasi kedua adalah aplikasi *frontend* yang berfungsi untuk monitoring kondisi kelembaban tanah. Aplikasi ini dibuat dengan menggunakan bahasa pemrograman *JavaScript* dengan *framework* *Vue.js*. Aplikasi *frontend* ini terdiri dari halaman *home*, *login* dan *dashboard*. Pertama untuk halaman *home* pada aplikasi *frontend* dapat dilihat pada Gambar 14.



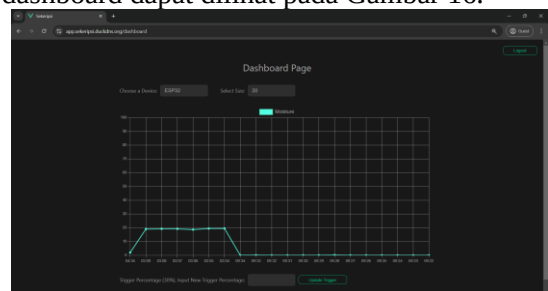
Gambar 14. Halaman *Home*

Kedua, untuk tampilan halaman login dapat dilihat pada Gambar 15.



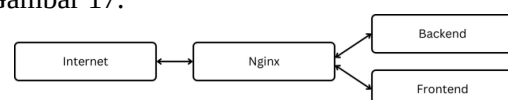
Gambar 15. Halaman *Login*

Ketiga, untuk tampilan dari halaman dashboard dapat dilihat pada Gambar 16.



Gambar 16. Halaman *Dashboard*

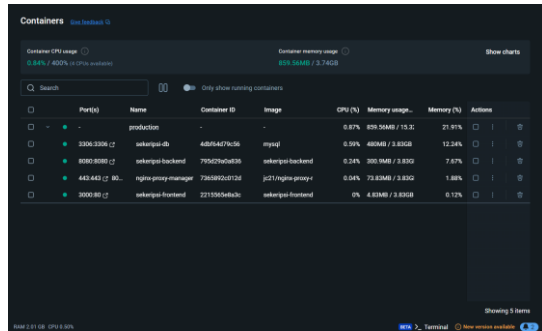
Kemudian kedua aplikasi baik itu *frontend* dan *backend* akan dibuat dalam bentuk *Docker Image*, yang akan dijalankan dalam bentuk sebuah *Docker Container*. Hal ini untuk mempermudah proses *deployment*. Kemudian aplikasi *frontend* dan *backend* akan dihubungkan dengan *Nginx Proxy Manager* untuk melakukan *reverse proxy*, sehingga *Nginx* yang akan menentukan *resource* apa yang harus diberikan kepada pengguna sesuai dengan *request* yang diberikan. *Nginx* disini berperan sebagai penengah yang bekerja sebagai *Gateway* menuju ke aplikasi *frontend* dan *backend*. Untuk proses kerja pada *web server* secara keseluruhan dapat dilihat pada Gambar 17.





Gambar 17. Alur Kerja Web Server

Kemudian *web server* akan dijalankan dalam satu *Docker Container* dengan *Network* yang sama sehingga aplikasi *backend* dan *frontend* dapat saling terhubung. Untuk tampilan dari hasil *web server* dapat dilihat pada Gambar 18.



Gambar 18. Tampilan Server pada Docker

3.3 Pengujian tanpa Kriptografi

Pengujian tanpa kriptografi dilakukan dengan mengirimkan data yang tidak melalui proses kriptografi. Tujuan dari pengujian ini adalah untuk melihat bentuk data yang dikirimkan ketika tidak melalui tahapan kriptografi. Untuk hasil dari pengujian ini dapat dilihat pada Gambar 19.



Gambar 19. Data tanpa Enkripsi

Dapat diperhatikan pada Gambar 19 bahwa data yang dikirimkan berupa *plaintext*, yang artinya semua informasi pada data yang dikirimkan dapat terbaca dengan jelas. Dari data ini dapat diperoleh beberapa informasi penting seperti *token* yang digunakan oleh ESP32 untuk berkomunikasi dengan API pada *web server*. Informasi lain seperti data sensor yang dikirim bernilai 3472, *response body* dari *web server* juga dapat terbaca dengan jelas yang berisi kode status 200 yang berarti data ini berhasil tersimpan pada *web server*. Hal ini merupakan bentuk proses transmisi data tanpa melakukan enkripsi atau menerapkan HTTPS. *Plaintext* ini merupakan vulnerabilitas pada proses transmisi data, yang dimana dengan semua informasi ini semua pihak yang dapat

memperoleh data ini baik secara sengaja ataupun tidak, juga dapat mengkonstruksi data serupa yang kemudian seakan-akan menjadi ESP32 dan mengirimkan data hasil buatan tersebut ke *web server*.

3.4 Pengujian dengan Kriptografi ECC

Pengujian dengan melalui kriptografi dilakukan dengan melakukan proses enkripsi terhadap data yang akan dikirim, sebelum data tersebut dikirimkan. Dapat diperhatikan pada Gambar 20, hasil dari enkripsi adalah data yang dikirimkan berupa *ciphertext*. Hal ini menunjukkan bahwa informasi dari data yang dikirimkan telah berhasil di enkripsi dan data ini tidak dapat terbaca jelas bahwa ESP32 sedang mengirimkan data kelembaban tanah kepada *web server* seperti halnya pada Gambar 19. Dengan demikian semua pihak yang mencoba melakukan serangan *man in the middle* untuk memperoleh data ini, tidak memiliki informasi atau cara untuk meretas proses transmisi data ini.



Gambar 20. Data dengan Enkripsi

Untuk mengenkripsi sebuah data dengan ECC, perlu dilakukan perhitungan untuk mengubah *plaintext* menjadi *ciphertext* dan sebaliknya. berikut adalah contoh sederhana langkah perhitungan menggunakan ECC dengan parameter $a = 3$; $b = 7$; $q = 3929$; $G = (4, 3012)$ kemudian nilai k yang disetujui antara *user A* dan *B* adalah $k = 1\%(K(x))$:

- 1) *User A* menghitung *key pair*:
 - a) Memilih *private key* $n_A = 16$
 - b) Menghitung *public key*:
$$P_A = 16 * (4,3012)P_A = (2703,3786)$$
- 2) *User B* menghitung *key pair*:
 - a) Memilih *private key* $n_B = 21$
 - b) Menghitung *public key*:
$$P_B = 21 * (4,3012)P_B = (93,84)$$



- 3) Kedua *user A* dan *B* masing masing menghitung *shared secret key* untuk menentukan nilai *k*:

a) *User A*:

$$K_A = n_A P_B$$

$$K_A = 16 * (93,84) K_A$$

$$= (3578,1348)$$

b) *User B*:

$$K_B = n_B P_A K_B$$

$$= 21 * (2703,3786) K_B$$

$$= (3578,1348)$$

- 4) Dari K_A dan K_B dapat ditentukan:

$$k = 1\%(3578)$$

$$k = 36$$

- 5) Misalkan *user A* ingin mengirimkan data berupa huruf *B* kepada *user B*, maka tentukan nilai *ASCII*:

$$B = 66$$

- 6) Angka 66 dapat di *encode* menjadi titik pada kurva dengan Persamaan (3):

$$x = (mk) + 1 \quad (3)x =$$

$$(66 \times 36) + 1$$

$$x = 2377$$

- 7) kemudian substitusikan nilai *x* pada langkah nomor 6 ke dalam persamaan awal kurva $y^2 \bmod 3929 = x^3 + 3x + 7 \bmod 3929$. Dan pada iterasi ke-2 dengan nilai: $x = 2378$ dapat ditentukan titik $P_m(x, y) = (2378, 2865)$ titik ini masih berupa *plain point* pada kurva.

- 8) *User A* melakukan enkripsi *plain point* menjadi *cipher point* dengan Persamaan (4):

$$C_m = \{ kG, (P_m + kP_B) \} \quad (4) C_m =$$

$$\{ 36(4,3012), ((2378,2865) + 36 * (93,84)) \} C_m =$$

$$\{ (2378,1064), ((2378,2865) + (1831,1688)) \} C_m =$$

$$\{ (2378,1064), (1224,832) \}$$

- 9) *User B* melakukan dekripsi *cipher point* menjadi *plain point* dengan Persamaan (5), perlu diingat bahwa baik itu Persamaan (5) ataupun Persamaan (6) akan menghasilkan nilai yang sama:

$$P_m = (P_m + kP_B) - n_B(kG) \quad (5) P_m =$$

$$(P_m + k(n_B)G) - n_B(kG) \quad (6) P_m =$$

$$(1224,832) - 21 * (2378,1064) P_m =$$

$$(1224,832) - (1831,1688)$$

- 10) Untuk melakukan operasi pengurangan titik $P - Q$ cukup dengan mengubah Q_y menjadi $-Q_y$ sehingga persamaan yang dihasilkan seperti pada langkah nomor 9 dapat dituliskan kembali dalam bentuk $(P_x, P_y) - (Q_x, Q_y) = (P_x, P_y) + (Q_x, -Q_y)$ [22], kemudian perhitungan menjadi.

$$P_m = (1224,832) + (1831, -1688) P_m$$

$$= (2378,2865)$$

- 11) Lakukan proses *decoding* dengan Persamaan (7):

$$m = (P_m(x) - 1)/k \quad (7)m =$$

$$(2378 - 1)/36$$

$$m = 66.03$$

Kemudian ambil bilangan bulat sehingga $m = 66$

- 12) Ubah bentuk *ASCII* ke bentuk asli dari data yang dikirimkan oleh *user A* sehingga dapat diperoleh:

$$66 = B$$

Kemudian pengujian kriptografi ini dilakukan sebanyak 30 kali. Tujuan dari pengujian ini adalah untuk melihat apakah data dapat terenkripsi secara konsisten melalui beberapa kali pengujian. Sesuai dengan perancangan perangkat lunak pada ESP32, data akan dikirimkan ke *web server* dengan jeda waktu setiap 10 detik. Kemudian untuk setiap data yang terkirim akan dianalisis dengan aplikasi *Wireshark*, kemudian akan dilihat bentuk datanya apakah data telah terenkripsi atau tidak. Untuk hasil pengujian dapat dilihat pada Tabel 2.

Tabel 2. Hasil Pengujian Enkripsi pada Sistem

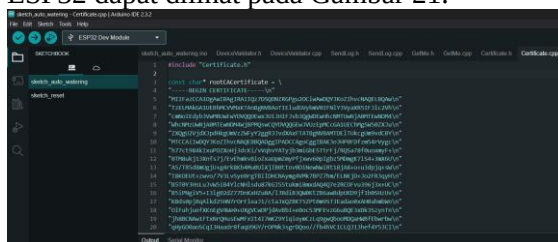
No.	Data	Status	Ekspektasi Output	Output	Keterangan
1.	2736 (34%)	Berhasil	Ciphertext	Ciphertext	Sesuai
2.	2716 (36%)	Berhasil	Ciphertext	Ciphertext	Sesuai
3.	2271 (56%)	Berhasil	Ciphertext	Ciphertext	Sesuai
4.	2432 (50%)	Berhasil	Ciphertext	Ciphertext	Sesuai
5.	2466 (49%)	Berhasil	Ciphertext	Ciphertext	Sesuai
6.	2554 (44%)	Berhasil	Ciphertext	Ciphertext	Sesuai
7.	2588 (42%)	Berhasil	Ciphertext	Ciphertext	Sesuai
8.	2610 (41%)	Berhasil	Ciphertext	Ciphertext	Sesuai
9.	2327 (56%)	Berhasil	Ciphertext	Ciphertext	Sesuai
10.	2389 (53%)	Berhasil	Ciphertext	Ciphertext	Sesuai
11.	2516 (46%)	Berhasil	Ciphertext	Ciphertext	Sesuai
12.	2559 (44%)	Berhasil	Ciphertext	Ciphertext	Sesuai
13.	2587 (42%)	Berhasil	Ciphertext	Ciphertext	Sesuai
14.	2619 (41%)	Berhasil	Ciphertext	Ciphertext	Sesuai
15.	2727 (35%)	Berhasil	Ciphertext	Ciphertext	Sesuai
16.	2638 (40%)	Berhasil	Ciphertext	Ciphertext	Sesuai
17.	2605 (41%)	Berhasil	Ciphertext	Ciphertext	Sesuai

18.	2672 (38%)	Berhasil	Ciphertext	Ciphertext	Sesuai
19.	2672 (38%)	Berhasil	Ciphertext	Ciphertext	Sesuai
20.	2671 (38%)	Berhasil	Ciphertext	Ciphertext	Sesuai
21.	2691 (37%)	Berhasil	Ciphertext	Ciphertext	Sesuai
22.	2705 (36%)	Berhasil	Ciphertext	Ciphertext	Sesuai
23.	2704 (36%)	Berhasil	Ciphertext	Ciphertext	Sesuai
24.	2731 (35%)	Berhasil	Ciphertext	Ciphertext	Sesuai
25.	2736 (34%)	Berhasil	Ciphertext	Ciphertext	Sesuai
26.	2737 (34%)	Berhasil	Ciphertext	Ciphertext	Sesuai
27.	2743 (34%)	Berhasil	Ciphertext	Ciphertext	Sesuai
28.	2767 (33%)	Berhasil	Ciphertext	Ciphertext	Sesuai
29.	2783 (32%)	Berhasil	Ciphertext	Ciphertext	Sesuai
30.	2775 (32%)	Berhasil	Ciphertext	Ciphertext	Sesuai

Data pada Tabel 2 menunjukkan hasil pengujian yang memperlihatkan hasil bentuk data yang di interupsi pada proses transmisi. Dapat diperhatikan bahwa dalam 30 kali pengujian, data yang diperoleh selama pengujian telah benar berupa *ciphertext*. Hal ini menunjukkan bahwa pengujian ini telah berhasil karena antara bentuk data yang diharapkan dan bentuk data yang diperoleh adalah sama.

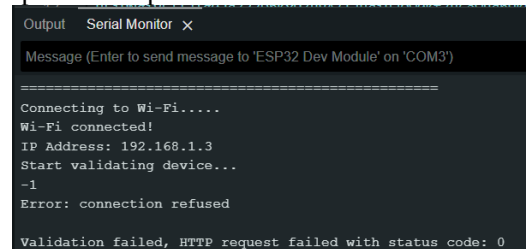
3.5 Pengujian Sertifikat Web Server

Sesuai dengan penjelasan sebelumnya bahwa salah satu mekanisme HTTPS dalam mengatasi penyerangan seperti *man in the middle* adalah dengan penerapan sistem sertifikat. Dengan protokol HTTPS, *web server* diharuskan untuk menyediakan sertifikat yang menunjukkan identitas dari *web server* yang sedang diakses. Sertifikat ini kemudian dapat digunakan oleh ESP32 untuk memastikan bahwa *server* yang sedang diakses merupakan *web server* yang tepat. Untuk implementasi sertifikat yang digunakan oleh *web server* pada ESP32 dapat dilihat pada Gambar 21.



Gambar 21. Sertifikat Server pada ESP32

Kemudian dengan informasi ini maka ESP32 dapat melakukan validasi apakah *web server* yang sedang diakses merupakan *web server* yang sesuai. Proses validasi ini dilakukan setiap ESP32 menghubungi *web server*. Untuk pengujian kasus dimana sertifikat yang diberikan oleh *web server* tidak sesuai dengan sertifikat yang sebenarnya pada ESP32 dapat dilihat pada Gambar 22.



Gambar 22. Pengujian Validasi yang Gagal

Dapat diperhatikan pada Gambar 22, ESP32 tidak akan mengirimkan data apapun apabila sertifikat yang diberikan *web server* tidak sesuai dengan sertifikat yang seharusnya dengan menampilkan *error: connection refused*. Hal ini memastikan bahwa ketika terjadi penyerangan seperti *man in the middle*, atau ESP32 diarahkan ke *web server* lain, maka ESP32 tidak akan mengirim data kepada *server* dari penyerang ataupun menerima data yang mungkin berisi program untuk mengubah pengaturan pada ESP32. Proses ini serupa dengan bagaimana *web browser* memberikan peringatan ketika pengguna hendak mengakses situs yang dianggap tidak aman ataupun tidak menyediakan sertifikat yang *valid*. Untuk mengetahui bahwa pengujian ini akan menghasilkan keluaran yang sama secara konsisten maka pengujian ini akan dilakukan sebanyak 30 kali. Untuk hasil pengujian dapat dilihat pada Tabel 3.

Tabel 3. Hasil Pengujian Validasi Sertifikat Web Server pada ESP32

Pengujian ke-n	Ekspektasi Output	Output	Keterangan
1.	Refused	Refused	Sesuai
2.	Refused	Refused	Sesuai
3.	Refused	Refused	Sesuai
4.	Refused	Refused	Sesuai
5.	Refused	Refused	Sesuai
6.	Refused	Refused	Sesuai
7.	Refused	Refused	Sesuai
8.	Refused	Refused	Sesuai
9.	Refused	Refused	Sesuai
10.	Refused	Refused	Sesuai
11.	Refused	Refused	Sesuai
12.	Refused	Refused	Sesuai



13.	Refused	Refused	Sesuai
14.	Refused	Refused	Sesuai
15.	Refused	Refused	Sesuai
16.	Refused	Refused	Sesuai
17.	Refused	Refused	Sesuai
18.	Refused	Refused	Sesuai
19.	Refused	Refused	Sesuai
20.	Refused	Refused	Sesuai
21.	Refused	Refused	Sesuai
22.	Refused	Refused	Sesuai
23.	Refused	Refused	Sesuai
24.	Refused	Refused	Sesuai
25.	Refused	Refused	Sesuai
26.	Refused	Refused	Sesuai
27.	Refused	Refused	Sesuai
28.	Refused	Refused	Sesuai
29.	Refused	Refused	Sesuai
30.	Refused	Refused	Sesuai

Data pada Tabel 3 menunjukkan hasil pengujian ketika ESP diberikan sertifikat yang tidak valid oleh *web server*. Dalam pengujian ini yang dipantau adalah bagaimana respon dari ESP32. Berdasarkan pengujian sebanyak 30 kali pada Tabel 3, maka dapat dikatakan ESP32 telah merespon situasi yang diberikan sesuai dengan respon yang diharapkan yaitu dimana ESP32 akan menolak untuk berkomunikasi dengan *web server* yang menyediakan sertifikat yang tidak valid kepada ESP32, sehingga pengujian ini telah berhasil.

4. KESIMPULAN

Berdasarkan hasil penelitian yang telah dilaksanakan, implementasi protokol HTTPS dengan algoritma kriptografi *Elliptic-Curve* dapat digunakan untuk mengamankan proses transmisi data kelembaban tanah oleh perangkat IoT ESP32 ke *web server*. Hal ini dibuktikan dengan memperhatikan apakah data yang dikirimkan dan data yang diterima bernilai sama, serta bentuk data ketika proses pengiriman berlangsung, sudah berupa *ciphertext*, yang berarti informasi penting dari transmisi data tidak dapat diperoleh. Adapun protokol ini, dapat mengamankan perangkat IoT dari akses yang tidak sah, hal ini dikarenakan perangkat IoT tidak akan mengirimkan data, ataupun menerima data apapun ketika proses validasi sertifikat pada protokol HTTPS gagal.

Selama proses penelitian berlangsung, terdapat beberapa hal yang menjadi keterbatasan dalam penelitian ini. Karena penelitian ini hanya berfokus kepada data yang diperoleh sensor, maka penelitian ini mengabaikan kondisi eksternal yang mungkin mempengaruhi hasil data yang diperoleh pada

sensor, misalnya cuaca ataupun kelembaban udara. Adapun terdapat keterbatasan lain seperti, karena setiap saat perangkat IoT perlu mengambil data ke *server* untuk memperoleh konfigurasi terbaru, ada kemungkinan dimana proses ini mungkin dapat mengganggu proses pengiriman data, apabila data yang dikirimkan berjumlah banyak sehingga dibutuhkan adanya mekanisme untuk mengatasi permasalahan ketika jumlah IoT yang digunakan lebih besar.

DAFTAR PUSTAKA

- [1] X. Mu and M. F. Antwi-Afari, "The applications of Internet of Things (IoT) in industrial management: a science mapping review," *Int. J. Prod. Res.*, vol. 62, no. 5, pp. 1928–1952, Mar. 2024, doi: 10.1080/00207543.2023.2290229.
- [2] M. Ammar, G. Russello, and B. Crispo, "Internet of Things: A survey on the security of IoT frameworks," *J. Inf. Secur. Appl.*, vol. 38, pp. 8–27, Feb. 2018, doi: 10.1016/j.jisa.2017.11.002.
- [3] A. Morchid, R. El Alami, A. A. Raezah, and Y. Sabbar, "Applications of internet of things (IoT) and sensors technology to increase food security and agricultural Sustainability: Benefits and challenges," *Ain Shams Eng. J.*, vol. 15, no. 3, p. 102509, Mar. 2024, doi: 10.1016/j.asej.2023.102509.
- [4] J. P. Kleinhans, "Internet of Insecure Things," *Stift. Neue Verantwort.*, 2017.
- [5] Z. Vahdati, S. Yasin, A. Ghasempour, and M. Salehi, "Comparison of ECC and RSA algorithms in IoT devices," *Vol.*, no. 16, 2005.
- [6] A. P. Felt, R. Barnes, A. King, C. Palmer, C. Bentzel, and P. Tabriz, "Measuring {HTTPS} Adoption on the Web," presented at the 26th USENIX Security Symposium (USENIX Security 17), 2017, pp. 1323–1338.
- [7] Z. Durumeric, J. Kasten, M. Bailey, and J. A. Halderman, "Analysis of the HTTPS certificate ecosystem," in *Proceedings of the 2013 conference on Internet measurement conference*, in IMC '13. New York, NY, USA: Association for Computing Machinery, Oct. 2013, pp. 291–304. doi: 10.1145/2504730.2504755.
- [8] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and T. Polk, "Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile," Internet Engineering Task Force, Request for Comments RFC 5280, May 2008. doi: 10.17487/RFC5280.
- [9] D. Mahto and D. K. Yadav, "RSA and ECC: A Comparative Analysis," vol. 12, no. 19, 2017.





- [10] L. J. Moleong, *Metodologi Penelitian Kualitatif (Edisi Revisi)*. Remaja Rosdakarya, 2016.
- [11] D. Assyakurrohim, D. Ikhrum, R. A. Sirodj, and M. W. Afgani, "Metode Studi Kasus dalam Penelitian Kualitatif," *J. Pendidik. Sains Dan Komput.*, vol. 3, no. 01, Art. no. 01, 2023, doi: 10.47709/jpsk.v3i01.1951.
- [12] D. Hercog, T. Lerher, M. Truntič, and O. Težak, "Design and Implementation of ESP32-Based IoT Devices," *Sensors*, vol. 23, no. 15, Art. no. 15, Jan. 2023, doi: 10.3390/s23156739.
- [13] H. Eller and A. Denoth, "A capacitive soil moisture sensor," *J. Hydrol.*, vol. 185, no. 1, pp. 137–146, Nov. 1996, doi: 10.1016/0022-1694(95)03003-4.
- [14] M. H. Massé, *REST API design rulebook: designing consistent RESTful Web Service Interfaces*. Beijing Köln: O'Reilly, 2012.
- [15] J. Aas *et al.*, "Let's Encrypt: An Automated Certificate Authority to Encrypt the Entire Web," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, London United Kingdom: ACM, Nov. 2019, pp. 2473–2487. doi: 10.1145/3319535.3363192.
- [16] J. Clark and P. C. van Oorschot, "SoK: SSL and HTTPS: Revisiting Past Challenges and Evaluating Certificate Trust Model Enhancements," in *2013 IEEE Symposium on Security and Privacy*, May 2013, pp. 511–525. doi: 10.1109/SP.2013.41.
- [17] H. AlMajed and A. AlMogren, "A Secure and Efficient ECC-Based Scheme for Edge Computing and Internet of Things," *Sensors*, vol. 20, no. 21, Art. no. 21, Jan. 2020, doi: 10.3390/s20216158.
- [18] T. Pramiyati, J. Jayanta, and Y. Yulnelly, "PERAN DATA PRIMER PADA PEMBENTUKAN SKEMA KONSEPTUAL YANG FAKTUAL (STUDI KASUS: SKEMA KONSEPTUAL BASISDATA SIMBUMIL)," *Simetris J. Tek. Mesin Elektro Dan Ilmu Komput.*, vol. 8, no. 2, Art. no. 2, Nov. 2017, doi: 10.24176/simet.v8i2.1574.
- [19] F. Jabnabillah, A. Aswin, and M. R. Fahlevi, "Efektivitas Situs Web Pemerintah Sebagai Sumber Data Sekunder Bahan Ajar Perkuliahan Statistika," *Sustain. J. Kaji. Mutu Pendidik.*, vol. 6, no. 1, pp. 59–70, Jun. 2023, doi: 10.32923/kjmp.v6i1.3373.
- [20] M. Ciesielska, K. W. Boström, and M. Öhlander, "Observation Methods," in *Qualitative Methodologies in Organization Studies: Volume II: Methods and Possibilities*, M. Ciesielska and D. Jemielniak, Eds., Cham: Springer International Publishing, 2018, pp. 33–52. doi: 10.1007/978-3-319-65442-3_2.
- [21] A. J. Kleinheksel, N. Rockich-Winston, H. Tawfik, and T. R. Wyatt, "Demystifying Content Analysis," *Am. J. Pharm. Educ.*, vol. 84, no. 1, p. 7113, Jan. 2020, doi: 10.5688/ajpe7113.
- [22] D. R. Brown, "Sec 1: Elliptic Curve Cryptography," *Certicom Res.*, vol. 2, 2009.

